



UNIVERSITÉ MOULAY ISMAIL
ÉCOLE SUPÉRIEURE DE TECHNOLOGIE
DÉPARTEMENT INFORMATIQUE
- MÉKNES -

MÉMOIRE DE STAGE DE FIN D'ETUDES

**Conception et développement d'une application web
basée sur le machine learning pour superviser et
commander un bras manipulateur à 6 degrés de
liberté (DDL).**

Présenté Par : BOUDA MOHAMED

Encadré au niveau de l'établissement par :

PR. IDRIS CHANA
PR. AZIZ BOUAZI
PR. ABDALLAH MAGHRAOUI HSAINI

Soutenu le 11/06/2024 devant le jury :

Pr. Idriss CHANA	UMI-EST-Meknès	Examineur
Pr. Morad HAJJI	UMI-EST-Meknès	Examineur

Année universitaire 2023-2024

Dédicaces :

Je dédie ce travail :

À mon cher père

Pour leur aide, leur soutien et leur patience, cette aventure n'aurait certainement pas existé sans vous !

À ma chère mère

En témoignage de notre éternelle reconnaissance, que Dieu vous protège et vous prête bonne santé et longue vie.

À mes frères

Qui ont toujours su me motiver même dans les moments de doute. Un grand merci pour leur encouragement et leur soutien moral.

Aux enseignants de l'ESTM

En témoignage de notre amour et de notre profonde admiration.

Remerciements

Tout d'abord, je tiens à exprimer ma gratitude envers **l'École Supérieure de Technologie de Meknès** pour m'avoir accordé l'opportunité de réaliser mon stage au sein de leur établissement.

Ensuite, je souhaite adresser mes remerciements les plus sincères à **M. Chana**, mon encadrant de stage, enseignant-chercheur à l'École Supérieure de Technologie de Meknès, premièrement pour m'avoir donné l'occasion de vivre cette nouvelle expérience dans le domaine de la robotique, et pour son accueil chaleureux, ses précieux conseils et la richesse des connaissances qu'il a partagées avec moi tout au long de mon stage. Je suis particulièrement reconnaissant pour la grande liberté qu'il m'a accordée durant mon stage ainsi que pour son soutien et sa disponibilité infaillible lorsque je rencontrais des difficultés.

Je souhaite également adresser mes remerciements à **Hafsa Khrouch**, doctorante à l'École Supérieure de Technologie de Meknès pour ses précieux conseils et son aide inestimable pendant mon stage.

Je voudrais également remercier toute l'équipe **IEVIA** pour leur esprit d'équipe, leur accueil et les compétences qu'ils ont partagées avec moi lors de mon stage.

Enfin, un grand merci à toutes les personnes qui ont contribué au succès de mon stage et qui m'ont aidé lors de la rédaction de ce rapport.

Résumé

En résumé, ce rapport est rédigé dans le cadre de mon stage de fin d'étude à l'École Supérieure de Technologie de Meknès. L'objectif principal de ce stage était de développer un système basé sur le machine learning pour superviser et commander un bras à 6 degrés de liberté (DDL). Le système repose sur un serveur web développé en Django pour envoyer les commandes au robot et simuler les commandes dans ROS, tout en exploitant un modèle basé sur le deep learning. De plus, une interface web a été développée en React.js.

Abstract

In summary, this report is written as part of my final year internship at the Higher School of Technology in Meknes. The main objective of this internship was to develop a system based on machine learning to supervise and control a 6-degree-of-freedom (DOF) robotic arm. The system relies on a web server developed in Django to send commands to the robot and simulate commands in ROS, while also utilizing a model based on deep learning. Additionally, a web interface was developed in React.js.

Table des matières

Remerciements :	2
Résumé	3
Table des matières	4
Table des figures	6
Introduction générale	8
1 Présentation de l'établissement d'accueil « EST de Meknès »	9
1.1 L'École Supérieure de Technologie de Meknès	9
1.2 Diplômes	9
1.3 Admission	10
2 Mission	11
2.1 Présentation	11
2.2 Origine du besoin	11
2.3 Expression fonctionnelle du besoin	11
2.3.1 Fonction principale	11
2.3.2 Fonctions complémentaires	12
3 Présentation des outils utilisés	13
3.1 Le robot Hercule 2000	13
3.1.1 Spécification du robot	14
3.1.2 Mode de fonctionnement	15
3.1.3 Format d'une instruction	16
3.1.4 Protocole de communication	17
3.2 ROS : Robot Operating System	17
3.2.1 Présentation de ROS	17
3.2.2 Pourquoi utiliser ROS ?	18
3.2.3 Outils de développement ROS	20
3.3 Google Colab	21
3.4 Convolutional Neural Networks	22
3.5 Bibliothèque Python	22
3.5.1 OpenCV (Open Source Computer Vision Library) :	22
3.5.2 TensorFlow :	22
3.5.3 Keras :	23
3.6 Technologie web	23
3.6.1 Django	23
3.6.2 React.js	23

3.6.3	La base de donne	23
4	Réalisation du projet	24
4.1	Importation du package URDF généré dans ROS et configuration pour la simulation avec Gazebo et MoveIt	24
4.1.1	Installation de ROS	24
4.1.2	Créer un espace de travail Catkin	26
4.1.3	Copier le package URDF exporter depuis SolidWorks dans le répertoire « src » de notre espace de travail « /hercule/src/ »	27
4.1.4	Configurer Le Fichier URDF Pour pour qu'il soit simulable	27
4.1.5	Créer et configurer un package MoveIt pour contrôler notre modèle virtuel	29
4.2	Création d'un modèle de reconnaissance d'image à l'aide de Tensorflow	31
4.2.1	Collecte des données	31
4.2.2	Construire un modèle de reconnaissance d'image en utilisant TensorFlow	32
4.3	Le développement côté serveur avec Django	42
4.4	La base de données	42
4.4.1	La gestion des utilisateur	43
4.4.2	service de robot	44
4.4.3	service de camera	45
4.5	Le développement d'interface web avec React.js	47
4.5.1	Conception de l'Application Web	47
4.5.2	Développement des Interfaces Utilisateurs	47
4.5.3	Authentification et autorisation	50
5	Etat du projet et perspectives envisageables	53
6	Difficultés rencontrées	54
	Conclusion générale	55
	Webographie	56

Table des figures

1	Robot hercule 2000	13
2	Configurez notre fichier source.list	24
3	Configurez nos clés	25
4	Installation de Ros-Noetic	25
5	Installation des packages python	25
6	Créez un répertoire de travail nommé "catkin_ws" contenant le dossier "src" à l'intérieur. Ensuite, initialisez le dossier de travail.	26
7	Construire l'espace de travail	26
8	Sourcer le fichier "setup.bash" automatiquement généré dans le répertoire catkin_ws/devel	26
9	La position zéro dans le robot réel et dans ROS	27
10	Lancement de notre modèle de robot dans Gazebo.	29
11	La base de données téléchargée depuis Kaggle contient 36 classes.	31
12	La base de données adaptée contient 3 classes.	32
13	Importation de jeu de données	32
14	Importation des Bibliothèques	33
15	Le prétraitement des images :	33
16	Le prétraitement des images pour la validation :	34
17	Initialisation du modèle séquentiel	34
18	définit une couche convolutionnelle	35
19	ajoute une couche de dropout	35
20	ajout deux nouvelles couches de convolution suivies d'une couche de max pooling à notre modèle CNN	35
21	ajout une couche de dropout à notre modèle	36
22	ajout une couche "Flatten" à notre modèle	36
23	ajout deux couches dense, une troisième couche de dropout avec un taux de 0,5	36
24	ajout une couche dense finale à notre modèle CNN	37
25	compiler notre modèle CNN avec l'optimiseur Adam	37
26	entraîner notre modèle CNN sur les données d'entraînement.	38
27	évaluer les performances de notre modèle CNN	38
28	enregistrer notre modèle	39
29	Visualisation de l'entraînement	39
30	visualiser les résultats de l'exactitude (accuracy) de l'entraînement.	40
31	Visualisation de l'exactitude de validation et des résultats.	40
32	Visualisation de l'exactitude de validation et des résultats.	41
33	Évaluation de l'ensemble de test	42
34	Le module conceptuel du module de données	43
35	le modèle DDL (Degré de Liberté)	44
36	le lancement de la camera	45

37	detection des objets	46
38	la page de connexion	48
39	La page d'ajout d'un utilisateur.	49
40	La page de liste des utilisateurs	49
41	Liste des commandes passées au robot	50
42	Les informations personnelles de l'utilisateur connecté.	50
43	Commander le robot :	51
44	Afficher le flux de la caméra et le résultat de détection.	51
45	Configuration des routes administratives :	52
46	Configuration des routes administratives des utilisateurs	52
47	la configuration des routes Dans notre application	52

Introduction générale

La robotique est un domaine scientifique qui traite de la conception, de la construction et de la programmation des machines capables d'exécuter des tâches de manière automatique ou semi-automatique, en fonction de programmes et d'algorithmes prédéterminés et adaptatifs. Ces machines, communément appelées « robots », peuvent être contrôlées par des humains ou fonctionner entièrement sous la supervision d'applications informatiques et d'algorithmes. Les robots sont en contact direct avec le monde réel et sont souvent utilisés pour remplacer les humains dans l'exécution de tâches monotones et répétitives. Ils peuvent être catégorisés en fonction de leur taille, de leur secteur d'intervention ou de leur finalité.

Dans l'optique de poursuivre mes études dans le domaine de l'informatique à l'École Supérieure de Technologie de Meknès (ESTM), j'ai eu l'opportunité de m'intégrer dans le laboratoire d'image pour effectuer mon stage de fin d'études et travailler sur le développement d'un système basé sur le machine learning pour superviser et commander un bras manipulateur à 6 degrés de liberté (DDL).

1 Présentation de l'établissement d'accueil « EST de Meknès »

1.1 L'École Supérieure de Technologie de Meknès

L'École Supérieure de Technologie de Meknès (ESTM) est un établissement public d'enseignement supérieur marocain situé sur la Route d'Agouray, Km 5, B.P. : 3103, Toulal - 50000. Elle a été créée en 1993 et a pour vocation de former des Techniciens Supérieurs en Génie Electrique, en Génie informatique, en Techniques de Management et en Techniques de Commercialisation et de Communication. L'ESTM étant un établissement de l'enseignement supérieur, elle réserve une place à la recherche scientifique et appliquée, au développement et au transfert de technologie en collaboration avec les établissements par le biais de l'encadrement thématique des stages et des projets de fin d'études et des cycles de formation continue.

La formation à l'ESTM est largement tournée vers l'établissement(entreprise), c'est pourquoi ses programmes ont le souci de répondre aux besoins et aux attentes de l'environnement économique et social et font appel à des méthodes pédagogiques en évolution permanente.

Les programmes d'enseignement comportent des Cours, des Travaux Pratiques, des Travaux Dirigés et des Travaux de Réalisation ainsi que des Stages et des Projets de Fin d'Etudes.

1.2 Diplômes

- Diplôme universitaire de technologie : Le Diplôme Universitaire de Technologie (DUT) est un diplôme national de l'enseignement supérieur marocain qui se prépare en deux ans, il est orienté vers l'insertion professionnelle des étudiants, mais propose aussi une solide formation théorique qui permet la poursuite d'études en licence professionnelle, école d'ingénieurs, facultés marocaines et

étrangères.

1.3 Admission

L'accès à l'EST Meknès est ouvert (selon la filière désirée) aux bacheliers des branches suivantes :

- Sciences Expérimentales : Filière Physique Chimie, Sciences de la Vie et de la Terre, Sciences Agronomiques
- Sciences Maths : Sciences Maths A, Sciences Maths B
- Techniques : Electronique, Electrotechnique, Filière Sciences et Technologies Electrique, Filière Sciences et Technologies Mécanique, Art Appliqué
- Filières gestion : Sciences Economiques, Filière Gestion Comptable, Technique de Gestion Administrative, Technique de Gestion Comptable

2 Mission

2.1 Présentation

Mon stage avait pour objectif de me familiariser avec ROS, un framework open-source utilisé dans le domaine de la robotique pour le développement de logiciels. Le projet principal consistait à réaliser une application web permettant de simuler et contrôler les mouvements du bras manipulateur Hercule 2000. Il s'agissait également de développer un modèle basé sur le machine learning pour détecter des objets et permettre au robot de se déplacer afin de prendre un objet détecté par la caméra et de le déplacer à un emplacement défini.

2.2 Origine du besoin

Parfois, lorsque le robot est commandé en mode réel, il effectue des mouvements qui peuvent être dangereux et risquent de le casser. Ainsi, dans un premier temps, il est nécessaire de développer un modèle 3D du robot sous SolidWorks et de l'implémenter dans ROS pour simuler le trajet du robot avant de le reproduire sur le robot réel. De plus, il est important de développer une application pour faciliter l'envoi des commandes au robot, au lieu de les envoyer via le terminal.

2.3 Expression fonctionnelle du besoin

2.3.1 Fonction principale

La fonction principale de ce projet consiste à implémenter le fichier URDF développé sous SolidWorks dans ROS, à le configurer avec MoveIt, et à le simuler dans Gazebo. Il s'agit également de développer un serveur web basé sur Django pour commander le robot et le simuler dans ROS, ainsi que de développer une application web en React.js pour simuler le robot dans ROS et envoyer des commandes au robot réel via un câble série.

2.3.2 Fonctions complémentaires

De plus, le projet consiste à développer un modèle basé sur le machine learning pour détecter des objets, en l'occurrence, un modèle qui détecte les fruits. Le robot devient alors capable de classifier les fruits, par exemple, en plaçant les bananes dans une boîte de bananes et les oranges dans une boîte d'oranges. Ensuite, il s'agit d'intégrer ce modèle avec notre serveur web Django et d'envoyer des commandes pour le simuler dans ROS, ainsi que de réaliser la simulation dans le monde réel.

3 Présentation des outils utilisés

Ce projet a demandé l'utilisation simultanée et complémentaire de plusieurs outils essentiels. Les descriptions détaillées de ces outils sont présentées dans ce chapitre.

3.1 Le robot Hercule 2000

Le robot hercule 2000 est un bras manipulateur à structure polaire articulé en 5 axes avec pince de préhension. Les actionneurs sont des moteurs à courant continu, limités en courant et associés à des potentiomètres de recopie. La transmission est assurée par un ensemble de courroies de type synchrones et tous les moteurs, sauf celui de la pince, sont situés dans le tronc du robot, assurant ainsi à l'ensemble une parfaite stabilité.



FIGURE 1 – Robot hercule 2000 .

3.1.1 Spécification du robot

Charge admissible : 1kg

Répétabilité : $\pm 0,5\text{mm}$

Résolution : $0,33^\circ$

Vitesse : $50^\circ/\text{s}$

Volume de travail : Sphère de 1,022m de diamètre

Déplacements :

Le robot Hercule 2000 est un bras manipulateur avec 5 degrés de liberté et une pince. Chaque axe du robot effectue des mouvements de rotation qui peuvent être symétriques ou non

- Base :Elle effectue des mouvements de rotations sur 320° répartis de manière symétrique dans les deux sens soit
 $-160^\circ \leq \text{BASE} \leq +160^\circ$.
- Épaule : Il effectue des mouvements de rotation sur 206° répartis de manière asymétrique :
 $-115^\circ \leq \text{ÉPAULE} \leq +91^\circ$.
- Il effectue des mouvements de rotations sur 206° répartis de manière asymétrique :
 $-124^\circ \leq \text{COUDE} \leq +82^\circ$.

- Tangage : Il effectue des mouvements rotatifs symétriques sur 180° :
 $-90^\circ \leq \text{TANGAGE} \leq +90^\circ$
- Roulis : Il effectue un mouvement de roulis symétrique sur 320° tel que
 $-160^\circ \leq \text{Roulis} \leq +160^\circ$.
- Pince de préhension : La pince est l'organe effecteur du bras manipulateur Hercule 2000. L'organe effecteur d'un robot est la partie qui lui permet de réaliser une tâche spécifique.
- Ouverture : 70mm max limitée
- Moteur à courant continu programmable en couple et en durée

Poids : 13kg

3.1.2 Mode de fonctionnement

Hercule permet 3 modes de fonctionnement distincts :

LE MODE LIE

Permet un dialogue interactif et bidirectionnel entre l'utilisateur et le robot. Pour y accéder l'utilisateur doit émettre le code ASCII de la lettre « L » vers le coffret de commande. Le robot est alors prêt à recevoir et exécuter une nouvelle instruction. Le dialogue ordinateur/robot est bidirectionnel jusqu'à la sélection d'un autre mode de fonctionnement.

LE MODE AUTONOME

Cette configuration permet d'utiliser Hercule comme un système indépendant, déconnecté de toute unité de programmation. Le robot exécute alors un programme téléchargé dans sa mémoire. Pour y accéder l'utilisateur émet le code ASCII de la lettre « A » vers le coffret de commande suivi du fichier (caractères ASCII) constituant le programme qui est alors placé en mémoire interne.

LE MODE APPRENTISSAGE

Permet la mise en mouvement du robot par l'intermédiaire d'un boîtier de télécommande relié au coffret par la prise d'Entrées/Sorties. Pour y accéder, l'utilisateur émet le code ASCII du caractère « \$ » vers le coffret via la liaison série pour configurer le robot en séquence d'apprentissage. Le coffret lui envoie en retour la position actuelle d'Hercule.

3.1.3 Format d'une instruction

Dans tous les cas de figure, le formatage des instructions doit respecter la syntaxe suivante : C+010,1 :20 où

C : Initiale de l'axe à mettre en œuvre (B, E, C, T, R)

+ : Sens de déplacement

010,1 : Position absolue en degrés dans l'espace de travail

« : » Séparateur automatique de paramètres

20 : Vitesse de déplacement ou durée d'une opération de serrage ou de desserrage

Il existe 2 types d'instructions :

— L'instruction simple, destinée à la commande d'un axe unique : B+511 <SP>
<LF>

— L'instruction composée, permettant la commande simultanée de tous les axes et correspondant à une succession d'instructions simples :

B+230E+208C+258T-40R+489 <SP> <LF>

Dans le cas d'un fichier en mode autonome, les instructions sont séparées par le caractère <SP> : B38E+22C+337T+511R+6<SP>

B+377E+22C+337T+511R+6 <SP>B201E+393C+460T+ 23R-16<SP>

B+67E+39C+107T+511R+0<SP><LF>

3.1.4 Protocole de communication

La communication entre le robot et l'ordinateur se fait par l'intermédiaire d'un câble convertisseur USB vers RS232. La programmation des mouvements s'effectue par l'intermédiaire de la liaison série ; la vitesse de transmission des données étant fixée à 9600 bauds. Le coffret de commande comprend et interprète des instructions écrites au format ASCII et identifiées par des opérateurs mnémoniques bien déterminés tels que :

- La sélection du mode de fonctionnement
- Le code de commande d'un axe moteur précis
- Le code de commande de la pince
- Le code de temporisation
- Les affectations des Entrées/Sorties

3.2 ROS : Robot Operating System

3.2.1 Présentation de ROS

ROS, abréviation de "Robot Operating System", est un ensemble d'outils et de bibliothèques logicielles conçus pour faciliter la création d'applications robotiques. Que vous ayez besoin de pilotes, d'algorithmes avancés ou d'outils de développement puissants, ROS offre une solution open source complète pour votre prochain projet de robotique. ROS est un système d'exploitation pour robots qui vous aide à développer facilement des robots et des applications robotiques en utilisant des outils et des bibliothèques open source. ROS fournit plusieurs fonctionnalités aux développeurs : abstraction du matériel, contrôle des périphériques de bas niveau, mise en œuvre de fonctionnalités couramment utilisées, transmission de messages entre les processus et gestions des packages installés.

3.2.2 Pourquoi utiliser ROS ?

Il n'est pas chose aisée de construire un robot. Pour le bon fonctionnement d'un robot, de nombreux éléments comme les moteurs, les capteurs, les logiciels et les batteries doivent travailler d'un commun accord sans discontinuité pour effectuer une tâche. ROS s'occupe de la définition des composants, des interfaces et des outils nécessaires à la construction de robots avancés. La majorité des robots sont composés d'acteurs (des dispositifs qui bougent), de capteurs (des dispositifs qui mesurent, détectent ou capturent des informations sur le monde) et de systèmes de contrôle (le cerveau du robot). ROS aide les développeurs à construire rapidement ces composants et à les connecter en utilisant les outils ROS. Ainsi au lieu de réinventer la roue, le développeur peut créer quelque chose de nouveau, le faire vite et bien en utilisant ROS. Par ailleurs l'utilisation de ROS présente de nombreux aspects positifs parmi lesquels on peut citer :

Communauté mondiale

Depuis plus de 10 ans, le projet ROS a créé un écosystème de logiciels pour la robotique qui a conduit à l'émergence d'une communauté mondiale de millions de développeurs et d'utilisateurs.

Réduire les délais de mise sur le marché

ROS fournit les outils, les bibliothèques et les capacités dont vous avez besoin pour développer vos applications robotiques, ce qui vous permet de consacrer plus de temps au travail qui est important pour votre établissement. Comme il s'agit d'un logiciel libre, vous avez la possibilité de décider où et comment utiliser ROS, ainsi que la liberté de l'adapter à vos besoins. De plus, ROS n'est pas exclusif, vous n'avez pas besoin de choisir entre ROS et une autre pile logicielle ; ROS s'intègre facilement à vos logiciels existants pour apporter ses outils à votre problème

Multi-domaine

ROS est une plateforme qui convient à un large éventail d'applications robotiques. Elle n'est pas spécifique à un type de robot en particulier et peut être utilisée pour des applications allant de l'intérieur à l'extérieur, de la maison à l'automobile, du sous-marin à l'espace, et du grand public à l'industrie.

Multi-plateforme

ROS 2 est supporté et testé sur Linux, Windows et MacOS, ainsi que sur diverses plateformes embarquées (via micro-ROS), ce qui permet un développement et un déploiement sans faille de l'autonomie des robots, de la gestion du backend et des interfaces utilisateur.

100 % open source

ROS est et sera toujours open-source, garantissant à la communauté mondiale un accès libre et sans entrave à un SDK robotique de haute qualité, le meilleur de sa catégorie et doté de toutes les fonctionnalités.

Favorable au commerce

ROS encourage vivement les dons de la communauté, mais ne les rend pas obligatoires. Tout utilisateur est libre de modifier ROS, de le combiner avec ses propres logiciels propriétaires et ceux d'autres personnes, puis de distribuer le résultat dans son propre produit propriétaire sans avoir à informer ROS.

Support Industriel

Des établissements du monde entier, grandes et petites, engagent leurs ressources pour contribuer à ROS 2 en tant que logiciel open source, et pour développer des produits à partir de ce logiciel

3.2.3 Outils de développement ROS

Cette partie étant relativement vaste nous nous limiterons aux outils utilisés dans notre projet.

Roscore

Le roscore est le serveur principal de ROS. Il s'agit d'une collection de nœuds et de programmes qui fournissent des services tels que la découverte de nœuds, la publication et la souscription de données, la résolution des noms de topic...etc

Il est lancé à l'aide de la commande roscore et doit être lancé avant de pouvoir utiliser d'autres outils ROS

NB : : Si vous utilisez la commande roslaunch, elle démarrera automatiquement roscore si elle détecte qu'il n'est pas déjà en cours d'exécution (à moins que l'argument `--wait` ne soit fourni)

Catkin

Catkin est un système de construction de paquets ROS qui permet de compiler et de générer des paquets ROS à partir du code source. Il permet de définir les dépendances entre les différents paquets.

Gazebo

Gazebo est un simulateur 3D, cinématique, dynamique et multirobot permettant de simuler des robots articulés dans des environnements complexes, intérieurs ou extérieurs, réalistes et en trois dimensions. Il est conçu pour être utilisé avec ROS et fournit une interface aux utilisateurs qui permet de tester et de valider le comportement du robot avant de le déployer dans le monde réel

Moveit and Rviz

MoveIt est une bibliothèque ROS (Robot Operating System) qui permet la planification de mouvements pour les robots. Elle fournit des outils pour la planification de

trajectoires, la manipulation d'objets, la détection de collision, la cinématique inverse, la simulation de mouvements, et bien plus encore aux développeurs.

Pour rendre tout cela interactif et visuel, il est souvent utilisé de concert avec Rviz ; un logiciel de visualisation 3D qui permet de visualiser en temps réel ce que le robot voit, pense et fait. Rviz permet de visualiser les données des capteurs du robot ce qui permet de comprendre comment ce dernier perçoit son environnement. Rviz est très sollicité dans le débogage et permet de visualiser les mouvements planifiés par MoveIt dans Rviz ce qui permet de vérifier et d'ajuster les trajectoires.

Rospy

rospy est une bibliothèque Python pour ROS. L'API client rospy permet aux programmeurs Python d'interagir avec les différents éléments de l'architecture ROS. En utilisant rospy un développeur peut créer des nœuds ROS en python pour communiquer avec d'autres nœuds ROS, publier et souscrire à des données de capteurs, et dans notre cas communiquer avec MoveIt et Rviz pour planifier des mouvements.

3.3 Google Colab

Dans le cadre de mon projet, Google Colab a été un outil essentiel pour l'entraînement et la construction du modèle de détection d'objets. Grâce à ses ressources de calcul puissantes, notamment les GPU disponibles, j'ai pu accélérer le processus d'entraînement du modèle. La plateforme conviviale de Colab a facilité la rédaction du code et l'exécution des tâches, tandis que l'intégration avec Google Drive a simplifié le stockage et le partage des données et des notebooks. Cette approche a permis un développement rapide et efficace du modèle, en optimisant les temps d'itération et en assurant une meilleure gestion des ressources de calcul disponibles.

3.4 Convolutional Neural Networks

Un réseau neuronal convolutif (CNN ou ConvNet) est une architecture réseau pour le Deep Learning qui apprend directement à partir des données.

Ce type de réseau est particulièrement efficace lorsqu'il s'agit de trouver des patterns dans des images afin de reconnaître des objets, des classes et des catégories. Les CNN peuvent aussi être efficaces pour la classification de données audio, de séries temporelles et de signaux.

3.5 Bibliothèque Python

3.5.1 OpenCV (Open Source Computer Vision Library) :

OpenCV est une bibliothèque open-source spécialisée dans le traitement d'images et la vision par ordinateur. Elle offre des fonctionnalités pour la manipulation d'images, la détection et la reconnaissance d'objets, le suivi d'objets en mouvement, la stéréovision, et bien plus encore. OpenCV est largement utilisée dans l'industrie et la recherche pour une variété d'applications, de la vision par ordinateur à la robotique.

3.5.2 TensorFlow :

TensorFlow est une bibliothèque open-source développée par Google, qui est largement utilisée pour le machine learning et le deep learning. Elle propose une infrastructure flexible pour la création et l'entraînement de réseaux neuronaux, ainsi que pour le déploiement de modèles dans divers environnements. TensorFlow offre une prise en charge native des CNN, des RNN (Réseaux de Neurones Récursifs), et d'autres architectures de réseaux neuronaux.

3.5.3 Keras :

Keras est une bibliothèque open-source haut niveau pour le deep learning, qui s'appuie sur TensorFlow (ou d'autres backends comme Theano ou Microsoft Cognitive Toolkit). Keras offre une interface simple et intuitive pour la création, l'entraînement et l'évaluation de modèles de deep learning. Elle permet aux développeurs de créer des réseaux neuronaux avec moins de code, tout en offrant une grande flexibilité pour la construction de modèles complexes.

3.6 Technologie web

3.6.1 Django

Django est un framework web open source en Python. Il a pour but de rendre le développement d'applications web simple et basé sur la réutilisation de code .

3.6.2 React.js

React JS est une technologie développée pour faciliter la création d'interface d'une application, elle permet notamment de créer des sites internet sous forme de Single Page Application (des sites internet qui sont constitués d'une seule page), en créant un DOM virtuel.

3.6.3 La base de donne

Dans ce projet, nous allons utiliser SQLite pour gérer nos données. SQLite est une base de données simple et légère, parfaite pour les petites et moyennes applications. Elle fonctionne sans serveur séparé, ce qui rend son installation et son utilisation très faciles. Cela nous permettra de gérer nos données de manière efficace et fiable.

4 Réalisation du projet

4.1 Importation du package URDF généré dans ROS et configuration pour la simulation avec Gazebo et MoveIt

Dans le cadre de notre projet, nous avons bénéficié d'un modèle URDF préalablement réalisé par d'anciens stagiaires du laboratoire d'images de l'École Supérieure de Technologie à Meknès.

4.1.1 Installation de ROS

Choix de la distribution ROS :

Au cours de ce stage, nous avons choisi d'utiliser comme distribution ROS-Noetic pour plusieurs raisons. Tout d'abord parce que c'est la dernière version de ROS 1 largement pris en charge, bénéficiant d'une large documentation et de ressources en lignes. De plus en travaillant avec ROS-Noetic nous pourrions nous familiariser avec les concepts de ROS 1 et mieux appréhender les différences avec ROS 2 lors de notre transition vers cette dernière.

Ainsi nous utiliserons donc Ubuntu 20.04 pour la suite de ce projet. C'est une version largement compatible avec ROS-Noetic

Installation de ROS-Noetic :

Pour installer ROS-noetic nous suivrons les instructions ci-après :

- Configurez notre fichier source.list :

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'
```

FIGURE 2 – Configurez notre fichier source.list

— Configurez nos clés :

```
sudo apt install curl # if you haven't already installed curl  
curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo apt-key add -
```

FIGURE 3 – Configurez nos clés

— Installation de Ros-Noetic :

```
sudo apt update
```

```
sudo apt install ros-noetic-desktop-full
```

FIGURE 4 – Installation de Ros-Noetic

— Instalation des packages python :

```
sudo apt install ros-noetic-desktop-full
```

FIGURE 5 – Instalation des packages python

4.1.2 Créer un espace de travail Catkin

Un espace de travail Catkin est un répertoire dans lequel nous pouvons organiser et construire nos packages ROS. C'est un environnement de développement spécifique à ROS qui facilite la création, la compilation et l'exécution de nos packages. Pour ce faire nous exécuterons ces commandes dans un terminal

- Mettre à jour nos packages Ubuntu : `sudo apt-get update`
- Créez un répertoire de travail nommé "catkin_ws" contenant le dossier "src" à l'intérieur. Ensuite, initialisez le dossier de travail.

```
$ mkdir -p ~/catkin_ws/src
$ cd ~/catkin_ws/src
$ catkin_init_workspace
```

FIGURE 6 – Créez un répertoire de travail nommé "catkin_ws" contenant le dossier "src" à l'intérieur. Ensuite, initialisez le dossier de travail.

- Construire l'espace de travail :

```
$ cd ~/catkin_ws/
$ catkin_make
```

FIGURE 7 – Construire l'espace de travail

- Sourcer le fichier "setup.bash" automatiquement généré dans le répertoire catkin_ws/devel

```
$ source devel/setup.bash
```

FIGURE 8 – Sourcer le fichier "setup.bash" automatiquement généré dans le répertoire catkin_ws/devel

4.1.3 Copier le package URDF exporter depuis SolidWorks dans le répertoire « src » de notre espace de travail « /hercule/src/ »

4.1.4 Configurer Le Fichier URDF Pour pour qu'il soit simulable

Comme je l'ai mentionné dans le premier chapitre, j'ai déjà un fichier URDF développé et configuré. J'ai simplement ajouté un entier dans ROS et apporté quelques modifications.

Définir les limites des joints :

Nous devons définir les limites de nos joints(articulations) pour qu'elles soient conformes aux limites d'Hercule. Trois points sont à prendre en compte à ce stade :

- Les limites du modèle réel :
- Le décalage entre les origines des repères du modèle réel et ceux du modèle virtuel ce qui induit qu'on doit retrancher ou ajouter une certaine rotation au modèle virtuel pour avoir exactement les mêmes positions que le modèle virtuel.

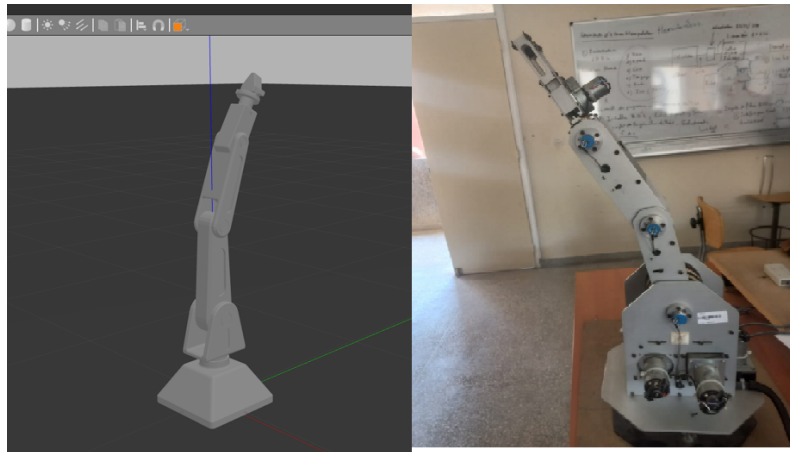


FIGURE 9 – La position zéro dans le robot réel et dans ROS

$$B_{virtuel} = B_{reel}$$

$$E_{virtuel} = E_{virtuel}$$

$$C_{virtuel} = C_{reel} - 71^\circ$$

$$T_{virtuel} = T_{reel} - 80^\circ$$

$$R_{virtuel} = R_{reel}$$

- Ajout de ± 5 degré aux limites des joints pour éviter les erreurs de « out of bounds » lors des conversions de degré à radians et vice versa. Cet ajout n'aura pas d'impacts significatifs car on empêchera l'utilisateur de dépasser les valeurs maximales d'Hercule dans notre systeme

Construire notre espace de travail catkin :

A cette étape nous lancerons les instructions nécessaires pour compiler notre package URDF

- `cd /catkin_ws`
- `source devel/setup.bash`
- `catkin build`
- `source devel/setup.bash`

Lancer notre robot dans Gazebo pour vérifier notre configuration :

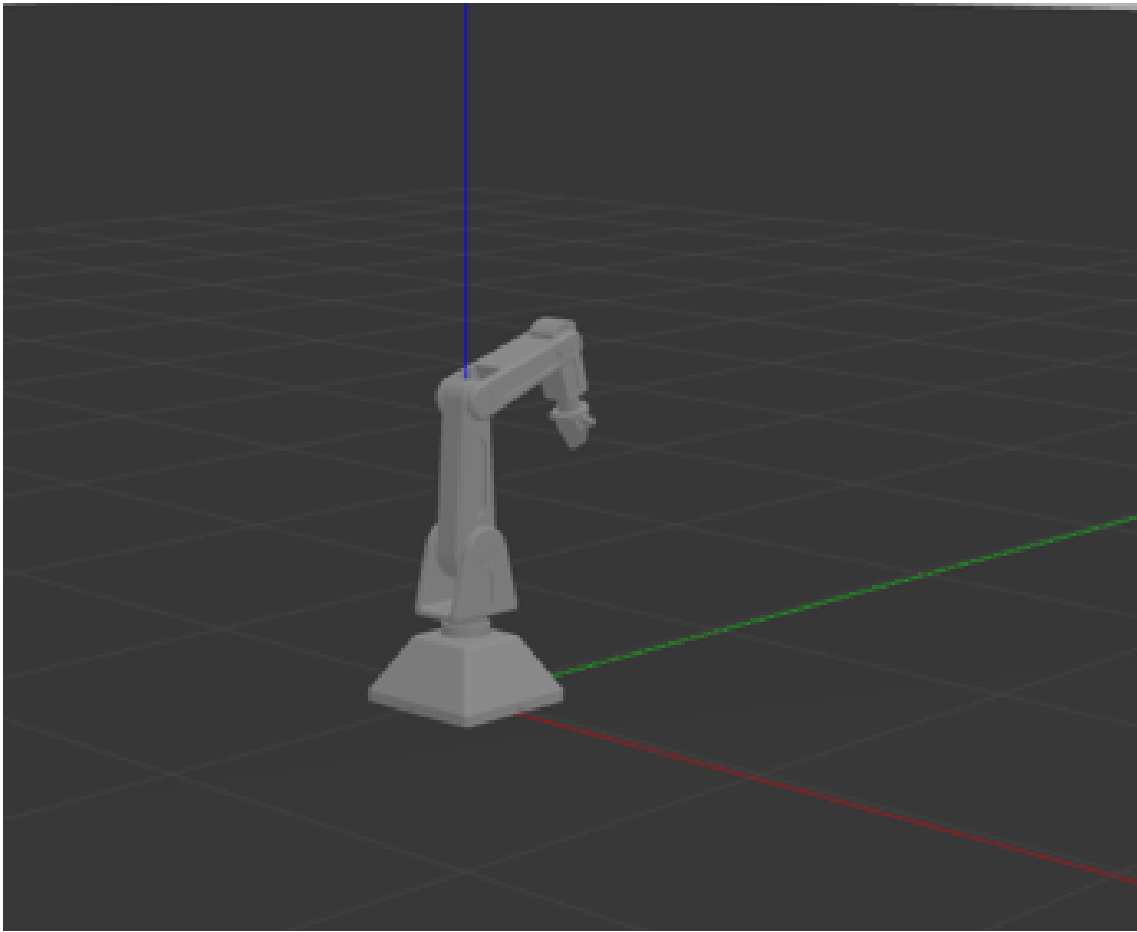


FIGURE 10 – Lancement de notre modèle de robot dans Gazebo.

4.1.5 Créer et configurer un package MoveIt pour contrôler notre modèle virtuel

MoveIt est un logiciel open source installé par défaut avec n'importe quelle distribution de ROS. Il permet de créer un des packages qui seront utilisés pour manipuler n'importe quel robot dans ROS

Créer un package MoveIt en utilisant MoveIt Setup Assistant :

- Aller dans notre espace
`cd /catkin_ws`

- Construire notre espace de travail si ce n'est pas déjà fait :
`catkin build`
- Sourcer le fichier `setup.bash` :
`source devel/setup.bash`
- Choisir `create new package` et charger notre fichier `robot_arm_urdf`
`roslaunch moveit_setup_assistant setup_assistant.launch`
- Dans 'Planning Groups', nous allons créer deux groupes :
 - Axes : pour le corps de le bras robotique .
 - pince : pour la pince de le bras robotique .
- Aller dans la section « Configuration Files » et enregistrer le package dans le répertoire `src` de notre espace Catkin

Lancer notre fichier hercule pour démarrer la simulation du robot avec Rviz et Gazebo :

- Dans notre espace de travail, sourcer le fichier `setup.bash` :
`source devel/setup.bash`
- Construire les packages :
`catkin build`
- Sourcer à nouveau le fichier `setup.bash` :
`source devel/setup.bash`
- Lancer notre simulation
`roslaunch moveit_hercule_robot hercule.launch`

Notre modèle est parfaitement importé et configuré dans ROS à ce stade. Maintenant, nous allons développer un serveur web Django pour commander le robot.

4.2 Création d'un modèle de reconnaissance d'image à l'aide de Tensorflow

4.2.1 Collecte des données

Dans le cadre de mon stage, j'ai utilisé une base de données disponible sur Kaggle, qui offre une collection riche et variée d'images d'aliments. Cette base de données contient 36 dossiers distincts, chacun renfermant des images spécifiques à un type d'aliment. En ce qui concerne les légumes, la base de données couvre une large gamme d'espèces. Cette diversité fait de cette base de données une ressource précieuse pour les projets en apprentissage automatique et en vision par ordinateur, ainsi que pour toute application nécessitant la reconnaissance et la classification des aliments.

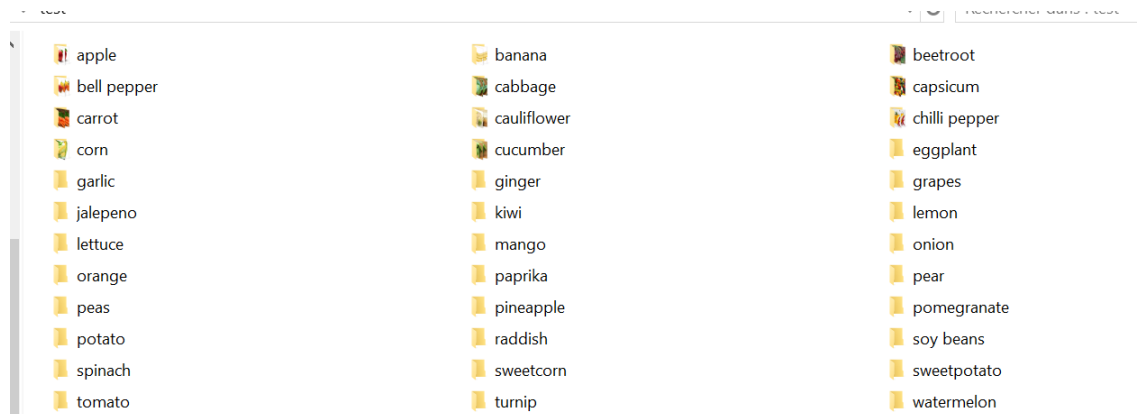


FIGURE 11 – La base de données téléchargée depuis Kaggle contient 36 classes.

Ensuite, j'ai ajusté la base de données pour ne garder que trois types de fruits : la banane, l'orange et la pomme. Cette étape m'a permis de me concentrer sur un ensemble plus restreint et plus facile à gérer pour mon projet. En travaillant uniquement avec ces trois fruits, j'ai simplifié le processus de reconnaissance et de classification, ce qui a rendu mon travail plus efficace et précis.

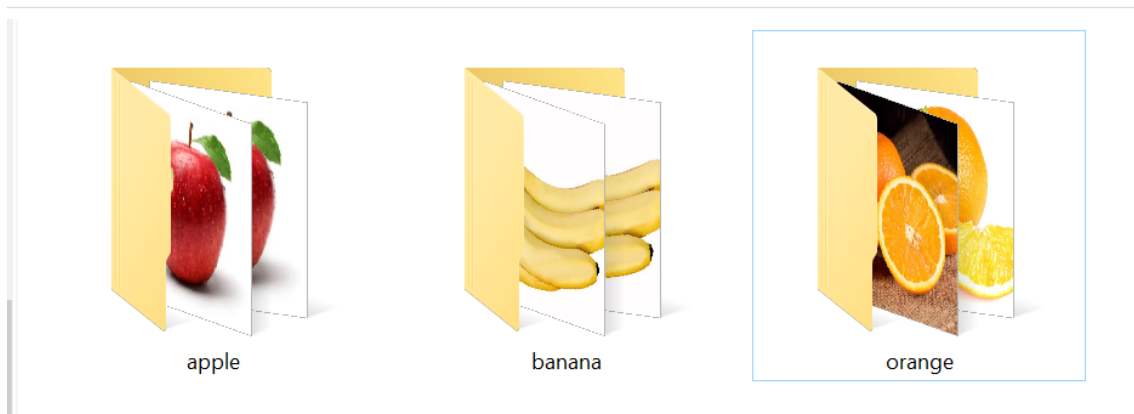


FIGURE 12 – La base de données adaptée contient 3 classes.

4.2.2 Construire un modèle de reconnaissance d'image en utilisant TensorFlow

Importation de jeu de données :

Pour développer le modèle de reconnaissance et de classification des fruits, j'ai utilisé Google Colab en conjonction avec Google Drive pour stocker la base de données d'images déjà collectée sur Kaggle .

```
from google.colab import drive
drive.mount('/content/drive')
```

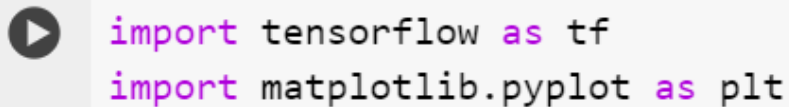
Mounted at /content/drive

FIGURE 13 – Importation de jeu de données

Importation des Bibliothèques :

Pendant mon stage, j'ai eu l'opportunité d'explorer et de développer ce modèle en utilisant deux bibliothèques essentielles : TensorFlow et Matplotlib.pyplot. TensorFlow, une bibliothèque open-source de pointe pour le machine learning et le deep learning, a été l'outil central pour la construction et l'entraînement du modèle. Sa flexibilité et sa puissance m'ont permis d'implémenter des architectures complexes et d'optimiser

les performances du modèle. En parallèle, l'utilisation de Matplotlib.pyplot a été cruciale pour visualiser les résultats, les tendances et les performances du modèle. Cette combinaison de bibliothèques a non seulement facilité le développement du modèle, mais a également fourni des outils essentiels pour analyser et interpréter ses résultats.

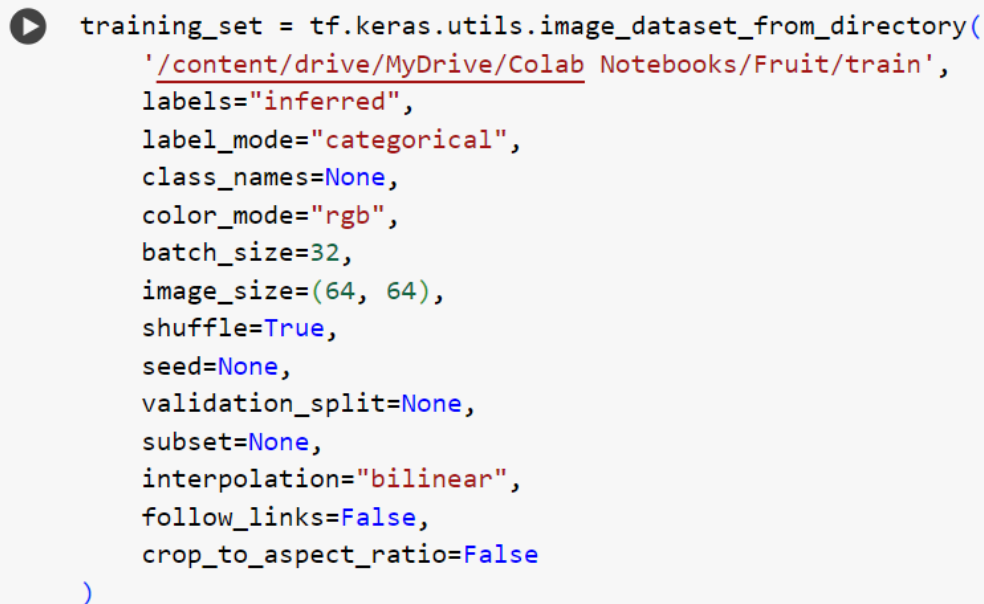


```
import tensorflow as tf
import matplotlib.pyplot as plt
```

FIGURE 14 – Importation des Bibliothèques

Le prétraitement des données :

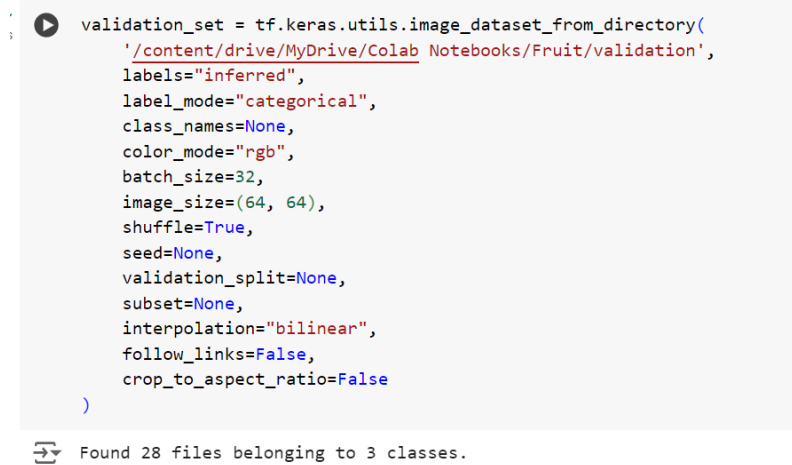
— Le prétraitement des images :



```
training_set = tf.keras.utils.image_dataset_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Fruit/train',
    labels="inferred",
    label_mode="categorical",
    class_names=None,
    color_mode="rgb",
    batch_size=32,
    image_size=(64, 64),
    shuffle=True,
    seed=None,
    validation_split=None,
    subset=None,
    interpolation="bilinear",
    follow_links=False,
    crop_to_aspect_ratio=False
)
```

FIGURE 15 – Le prétraitement des images :

— Le prétraitement des images pour la validation :



```
validation_set = tf.keras.utils.image_dataset_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Fruit/validation',
    labels="inferred",
    label_mode="categorical",
    class_names=None,
    color_mode="rgb",
    batch_size=32,
    image_size=(64, 64),
    shuffle=True,
    seed=None,
    validation_split=None,
    subset=None,
    interpolation="bilinear",
    follow_links=False,
    crop_to_aspect_ratio=False
)
```

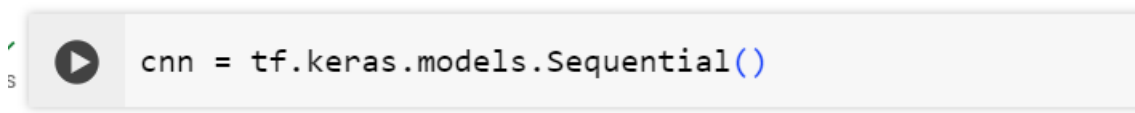
Found 28 files belonging to 3 classes.

FIGURE 16 – Le prétraitement des images pour la validation :

Construction du Modèle :

Pour construire le modèle de réseau de neurones convolutionnel (CNN), j'ai suivi les étapes suivantes. Voici le code en Python :

— Initialisation du modèle séquentiel :



```
cnn = tf.keras.models.Sequential()
```

FIGURE 17 – Initialisation du modèle séquentiel

— définit une couche convolutionnelle suivie d'une autre couche de convolution et d'une couche de max pooling dans un réseau de neurones convolutionnel (CNN) utilisant TensorFlow et Keras.

```
▶ cnn.add(tf.keras.layers.Conv2D(filters=32, kernel_size=3, padding='same', activation='relu', input_shape=[64, 64, 3]))  
cnn.add(tf.keras.layers.Conv2D(filters=32, kernel_size=3, activation='relu'))  
cnn.add(tf.keras.layers.MaxPool2D(pool_size=2, strides=2))
```

FIGURE 18 – définit une couche convolutionnelle

- ajoute une couche de dropout à notre modèle. Le dropout est une technique de régularisation utilisée pour réduire le surapprentissage (overfitting) en forçant les unités du réseau à apprendre de manière plus robuste et indépendante.

```
[7] cnn.add(tf.keras.layers.Dropout(0.25))
```

FIGURE 19 – ajoute une couche de dropout

- Ensuite, nous allons ajouter deux nouvelles couches de convolution suivies d'une couche de max pooling à notre modèle CNN :

```
▶ cnn.add(tf.keras.layers.Conv2D(filters=64, kernel_size=3, padding='same', activation='relu'))  
cnn.add(tf.keras.layers.Conv2D(filters=64, kernel_size=3, activation='relu'))  
cnn.add(tf.keras.layers.MaxPool2D(pool_size=2, strides=2))
```

FIGURE 20 – ajout deux nouvelles couches de convolution suivies d'une couche de max pooling à notre modèle CNN

- Ensuite, nous allons ajouter une couche de dropout à notre modèle :

```
[9] cnn.add(tf.keras.layers.Dropout(0.25))
```

FIGURE 21 – ajout une couche de dropout à notre modèle

- Ensuite, nous allons ajouter une couche "Flatten" à notre modèle. Cette couche est utilisée pour convertir les données en un format unidimensionnel (vecteur) avant de les passer à une couche dense pour la classification :

```
[10] cnn.add(tf.keras.layers.Flatten())
```

FIGURE 22 – ajout une couche "Flatten" à notre modèle

- Ensuite, nous allons ajouter deux couches denses, une avec 512 unités et l'autre avec 256 unités. Chaque neurone est connecté à tous les neurones de la couche précédente, et la fonction d'activation ReLU est utilisée pour introduire de la non-linéarité dans les données. Ensuite, nous ajoutons une troisième couche de dropout avec un taux de 0,5 (50%) pour supprimer 50% des unités de sortie de la couche précédente lors de l'entraînement.

```
[11] cnn.add(tf.keras.layers.Dense(units=512,activation='relu'))
```

```
[12] cnn.add(tf.keras.layers.Dense(units=256,activation='relu'))
```

```
[13] cnn.add(tf.keras.layers.Dropout(0.5))
```

FIGURE 23 – ajout deux couches dense, une troisième couche de dropout avec un taux de 0,5

- Ensuite, nous allons ajouter une couche dense finale à notre modèle CNN, composée de 3 neurones, correspondant aux 3 classes du modèle (banane, orange et pomme) :

```
 cnn.add(tf.keras.layers.Dense(units=3,activation='softmax'))
```

FIGURE 24 – ajout une couche dense finale à notre modèle CNN

Phase de compilation et d'entraînement :

Dans cette étape, nous allons compiler notre modèle CNN avec l'optimiseur Adam, la fonction de perte categorical_crossentropy et la métrique de précision (accuracy).

```
[ ] cnn.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['accuracy'])
```

```
 cnn.summary()
```

 Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
conv2d (Conv2D)	(None, 64, 64, 32)	896
conv2d_1 (Conv2D)	(None, 62, 62, 32)	9248
max_pooling2d (MaxPooling2D)	(None, 31, 31, 32)	0
dropout (Dropout)	(None, 31, 31, 32)	0
conv2d_2 (Conv2D)	(None, 31, 31, 64)	18496
conv2d_3 (Conv2D)	(None, 29, 29, 64)	36928

FIGURE 25 – compiler notre modèle CNN avec l'optimiseur Adam

Ensuite, nous allons entraîner notre modèle CNN sur les données d'entraînement.

```

training_history = cnn.fit(x=training_set, validation_data=validation_set, epochs=32)

```

```

Epoch 1/32
7/7 [=====] - 46s 1s/step - loss: 96.8823 - accuracy: 0.3255 - val_loss: 2.9323 - val_accuracy: 0.5357
Epoch 2/32
7/7 [=====] - 11s 669ms/step - loss: 2.7601 - accuracy: 0.3491 - val_loss: 1.0808 - val_accuracy: 0.4643
Epoch 3/32
7/7 [=====] - 12s 742ms/step - loss: 1.1202 - accuracy: 0.3443 - val_loss: 1.0938 - val_accuracy: 0.3571
Epoch 4/32
7/7 [=====] - 12s 1s/step - loss: 1.0821 - accuracy: 0.3302 - val_loss: 1.0869 - val_accuracy: 0.4286
Epoch 5/32
7/7 [=====] - 10s 664ms/step - loss: 1.0430 - accuracy: 0.4434 - val_loss: 0.9265 - val_accuracy: 0.6071
Epoch 6/32
7/7 [=====] - 12s 660ms/step - loss: 0.9065 - accuracy: 0.5425 - val_loss: 0.8222 - val_accuracy: 0.7857
Epoch 7/32
7/7 [=====] - 12s 661ms/step - loss: 0.8101 - accuracy: 0.6557 - val_loss: 0.7850 - val_accuracy: 0.6071
Epoch 8/32
7/7 [=====] - 10s 690ms/step - loss: 0.8001 - accuracy: 0.6321 - val_loss: 0.7498 - val_accuracy: 0.6429

```

FIGURE 26 – entraîner notre modèle CNN sur les données d'entraînement.

Évaluation du modèle :

Dans cette étape, nous allons évaluer les performances de notre modèle CNN sur les ensembles d'entraînement et de validation en termes de perte (loss) et de précision (accuracy).

```

[ ] train_loss, train_acc = cnn.evaluate(training_set)
    print('Training accuracy:', train_acc)

```

```

7/7 [=====] - 9s 132ms/step - loss: 0.1857 - accuracy: 0.9575
Training accuracy: 0.9575471878051758

```

```

val_loss, val_acc = cnn.evaluate(validation_set)
    print('Validation accuracy:', val_acc)

```

```

1/1 [=====] - 1s 941ms/step - loss: 0.5772 - accuracy: 0.8571
Validation accuracy: 0.8571428656578064

```

FIGURE 27 – évaluer les performances de notre modèle CNN

Enregistrement du modèle :

Dans cette étape, nous allons enregistrer notre modèle sous le nom "fruit_detection" avec le format .h5.

```
[ ] cnn.save('fruit_detection.h5')
```

```
[ ] training_history.history
```

```
➡ {'loss': [96.88233184814453,  
            2.760101079940796,  
            1.1201671361923218,  
            1.082136869430542,  
            1.043042540550232,  
            0.0065374000167817
```

FIGURE 28 – enregistrer notre modèle

Visualisation de l'exactitude (Accuracy Visualization) :

— Visualisation de l'entraînement :

```
✓ [31] epochs = [i for i in range(1,33)]  
0s plt.plot(epochs,training_history.history['accuracy'],color='red')  
    plt.xlabel('No. of Epochs')  
    plt.ylabel('Training Accuracy')  
    plt.title('Visualization of Training Accuracy Result')  
    plt.show()
```

FIGURE 29 – Visualisation de l'entraînement .

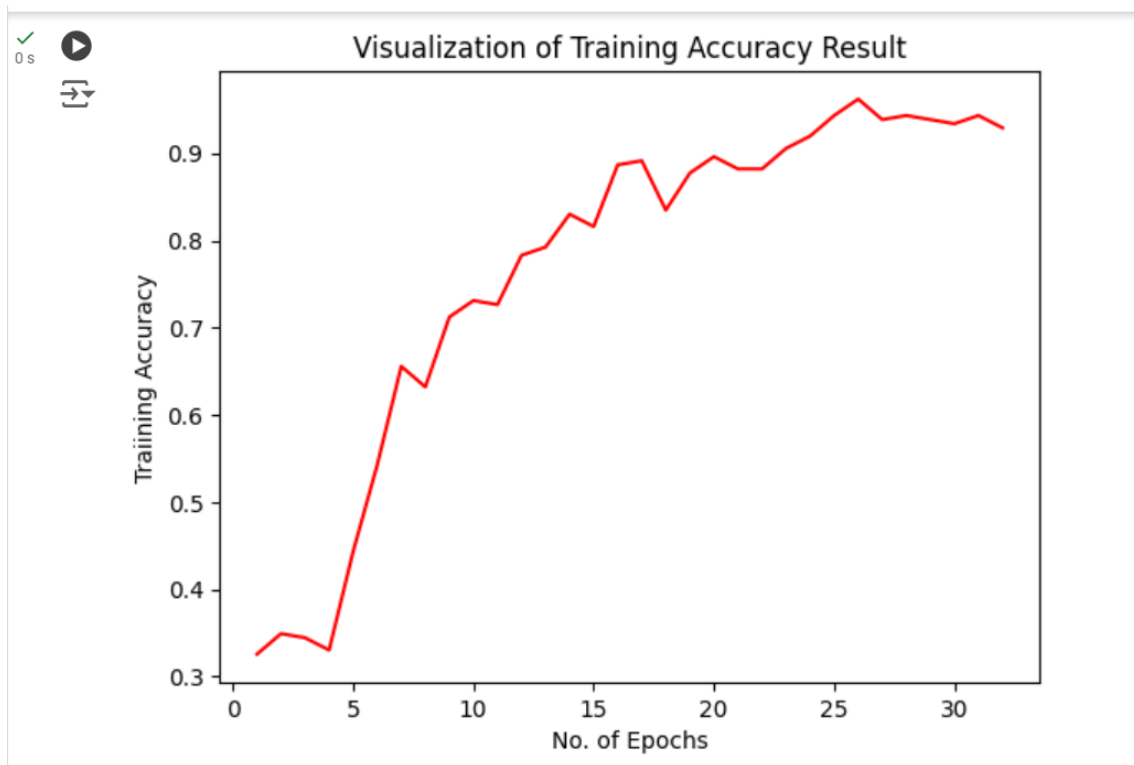


FIGURE 30 – visualiser les résultats de l'exactitude (accuracy) de l'entraînement.

— Exactitude de la validation (Validation Accuracy) :



FIGURE 31 – Visualisation de l'exactitude de validation et des résultats.

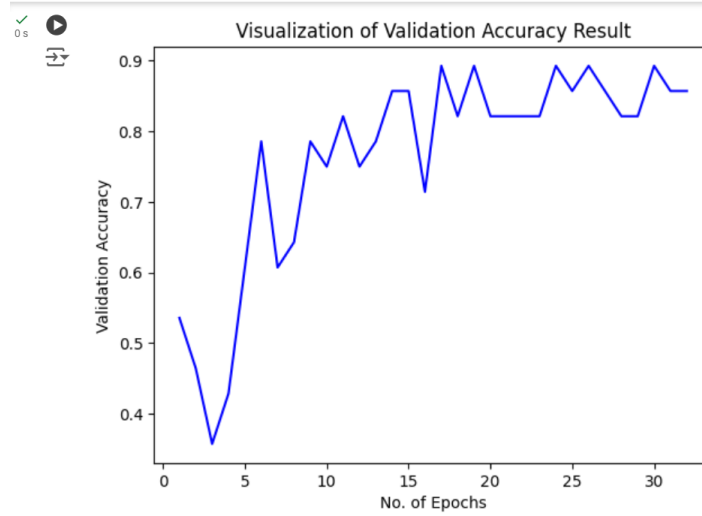


FIGURE 32 – Visualisation de l'exactitude de validation et des résultats.

— Évaluation de l'ensemble de test :



```

test_set = tf.keras.utils.image_dataset_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Fruit/test',
    labels="inferred",
    label_mode="categorical",
    class_names=None,
    color_mode="rgb",
    batch_size=32,
    image_size=(64, 64),
    shuffle=True,
    seed=None,
    validation_split=None,
    subset=None,
    interpolation="bilinear",
    follow_links=False,
    crop_to_aspect_ratio=False
)

Found 29 files belonging to 3 classes.

[ ] test_loss, test_acc = cnn.evaluate(test_set)
    print('Test accuracy:', test_acc)

1/1 [=====] - 7s 7s/step - loss: 0.3756 - accuracy: 0.8966

```

FIGURE 33 – Évaluation de l'ensemble de test .

4.3 Le développement côté serveur avec Django

Dans notre projet, nous utilisons une architecture client-serveur. Le serveur est développé avec Django. Django gère les services de gestion des utilisateurs, la simulation des commandes dans ROS, ainsi que leur envoi au robot réel. Il s'occupe également du lancement de la caméra et de la détection des objets.

4.4 La base de données

Dans notre projet, nous allons créer une base de données avec trois tables : une pour enregistrer les utilisateurs de notre application, une autre pour les commandes des valeurs de degrés de liberté du robot pour effectuer les mouvements, et une troisième pour les utilisateurs. Nous prévoyons de séparer les services de notre API entre ceux destinés aux administrateurs et ceux destinés aux utilisateurs normaux.

modèle conceptuel des données (MCD) :

la figure ci dessous illustre le diagramme du module conceptuel du module de données

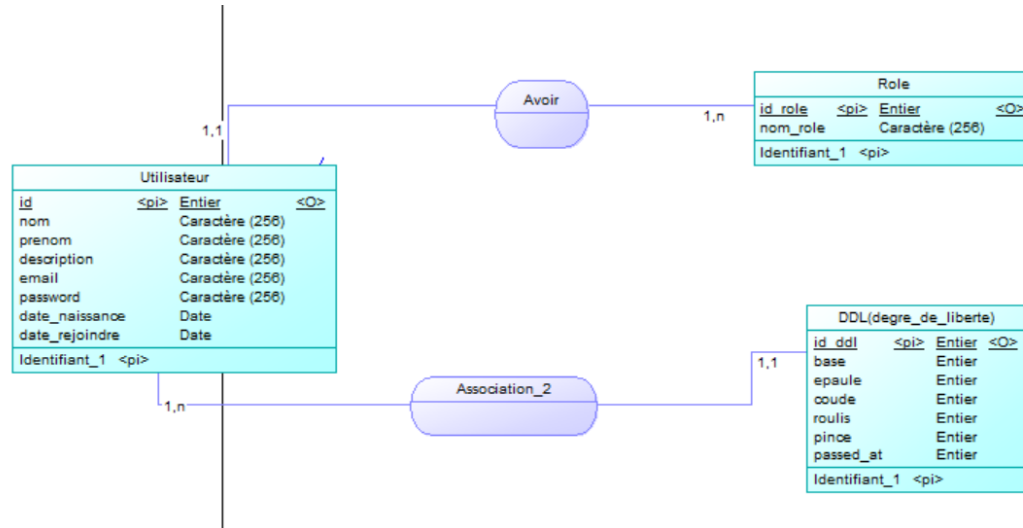


FIGURE 34 – Le module conceptuel du module de données

4.4.1 La gestion des utilisateur

dans notre api django nous allons developper des web services pour gestion des utilisateur dans notre applications web , ci deci la description des web services pour la gestion des utilisateur :

Ajouter un utilisateur

Ce service permet d'ajouter un nouvel utilisateur à notre application.

Modifier un utilisateur

Ce service permet de modifier les informations d'un utilisateur.

Supprimer un utilisateur

Ce service permet de supprimer un utilisateur de notre système.

Bloquer un utilisateur

Ce service permet de mettre un utilisateur en liste noire. Il n'aura pas l'autorisation d'accéder aux services de notre application.

4.4.2 service de robot

Comme notre objectif de stage est de gérer les mouvements d'un robot, nous allons développer des services web pour la gestion des mouvements de notre robot.

le modèle DDL (Degré de Liberté) :

Nous allons utiliser un modèle nommé DDL pour enregistrer tous les mouvements du robot dans notre base de données.

```
class DDL(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    base = models.IntegerField()
    epaule = models.IntegerField()
    coude = models.IntegerField()
    roulis = models.IntegerField()
    tanguage = models.IntegerField()
    pince = models.IntegerField()
    created_at = models.DateTimeField(auto_now_add=True)
```

FIGURE 35 – le modèle DDL (Degré de Liberté)

Simuler dans ROS :

Le web service prend les valeurs des degrés de liberté et les simule dans ROS.

Contrôleur ROS pour Convertir et Simuler les Mouvements du Robot Réel :

Ce contrôleur récupère les valeurs actuelles des degrés de liberté du robot et les intègre dans une simulation ROS pour une représentation virtuelle synchronisée des mouvements réels du robot.

Contrôleur ROS pour l'Envoi de Commandes au Robot Réel :

Ce contrôleur permet d'envoyer des valeurs de degrés de liberté au robot réel pour les exécuter.

Mode automatique

Le mode automatique de l'interface graphique permet à l'utilisateur de créer une tâche automatisée en définissant un ensemble d'instructions dans un fichier. Ces instructions seront ensuite envoyées à Hercule. Une fois le fichier reçu, Hercule stockera les instructions en mémoire et les exécutera en boucle infinie jusqu'à ce que l'utilisateur arrête le processus.

4.4.3 service de camera

Cette étape consiste à utiliser un modèle de détection d'objets basé sur CNN, intégré avec un serveur web, et associé à une caméra réelle. Un contrôleur est ensuite utilisé pour déclencher la caméra et exécuter des mouvements préprogrammés en fonction des objets détectés.

le lancement de camera

Nous allons utiliser OpenCV pour lancer une caméra de détection en temps réel.



FIGURE 36 – le lancement de la camera

Charger le modèle :

Ensuite, nous allons charger le modèle de détection déjà défini pour l'utiliser avec la caméra en temps réel en utilisant OpenCV.

Détection des objets :

Ensuite, exécutez le modèle pour détecter les objets, et une fois qu'un objet est détecté, exécutez une liste de mouvements prédéfinis.

```
if len(classIds) != 0:
    for classId, confidence, box in zip(classIds.flatten(), confs.flatten(), bbox):
        label = classNames[classId - 1]
        if label.lower() in ['orange', 'banana']:
            print(label)
            cv2.rectangle(img, box, color=(0, 255, 0), thickness=2)
            cv2.putText(img, label, (box[0] + 10, box[1] + 20),
                        cv2.FONT_HERSHEY_COMPLEX, 1, (0, 255, 0), thickness=2)
```

FIGURE 37 – detection des objets

Retour de flux vidéo en temps réel :

Notre service de caméra transmet un flux vidéo en temps réel qui est affiché dans notre application web côté client. Grâce à cette intégration, les utilisateurs peuvent visualiser en direct ce que la caméra capture, offrant ainsi une expérience immersive et en temps réel. Cette fonctionnalité permet une surveillance en direct et une interaction en temps réel avec le contenu visuel, renforçant ainsi l'efficacité et la convivialité de notre application.

Notification en Temps Réel via Django Channels :

Notre serveur web utilise Django Channels pour permettre la communication en temps réel avec le client. Lorsque notre système détecte un objet et exécute un mouvement prédéfini, il envoie instantanément une notification au client via les canaux Django.

4.5 Le développement d'interface web avec React.js

Dans cette étape, nous allons développer une interface web pour exploiter les API et les services fournis par Django.

4.5.1 Conception de l'Application Web

Pour développer cette application, j'ai utilisé React.js, une bibliothèque JavaScript, pour créer des interfaces web interactives. React.js permet de construire des composants réutilisables, rendant l'application plus facile à maintenir et à faire évoluer. Grâce à sa gestion efficace des mises à jour, l'interface reste rapide et réactive. React.js s'intègre bien avec les API Django, facilitant ainsi la communication entre le front-end et le back-end. Avec React.js, nous avons pu développer des pages pour la gestion des utilisateurs, le contrôle des robots, la surveillance en direct via la caméra et la détection d'objets.

Intégration avec les API Django

Pour accéder aux API Django, j'ai utilisé Axios, un outil JavaScript côté client. Avec Axios, nous pouvons facilement envoyer des requêtes HTTP depuis notre application React.js vers les services Django. Cela nous permet de récupérer, créer, mettre à jour et supprimer des données de manière asynchrone. L'intégration avec React.js s'est déroulée sans accroc, offrant ainsi une expérience utilisateur fluide.

4.5.2 Développement des Interfaces Utilisateurs

Page de connexion :

Pour accéder à notre application, il faut se connecter en entrant le nom d'utilisateur et le mot de passe. L'application sera divisée en deux parties : une partie pour les administrateurs et une autre pour les utilisateurs normaux.

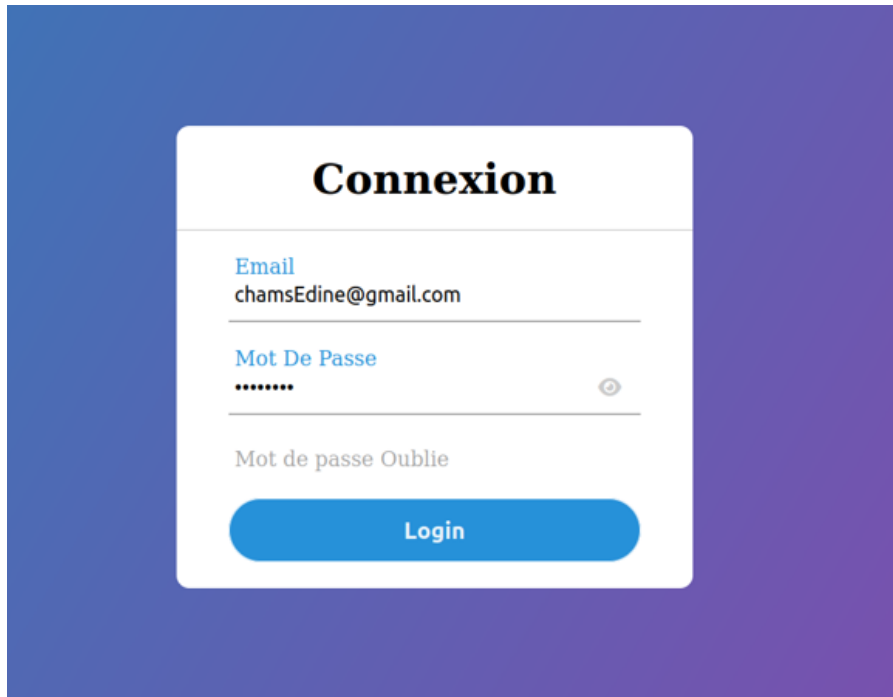


FIGURE 38 – la page de connexion

Les Pages De Admin :

- Ajouter Un utilisateur : Cette page permet à l'administrateur d'ajouter de nouveaux utilisateurs à notre système afin d'accéder aux services du robot.
- Liste des utilisateurs : Cette page permet à l'administrateur d'avoir la liste des utilisateurs dans notre système, avec la possibilité de bloquer un utilisateur pour qu'il n'accède pas aux services du robot.
- Liste des commandes passées au robot : Cette page offre la possibilité de voir les commandes passées au robot, que ce soit simulées dans ROS ou réelles. De plus, elle offre la possibilité de filtrer les commandes par utilisateur de l'application.

Les pages de l'utilisateur normal :

- Les informations personnelles de l'utilisateur connecté :

robot hercule 2000

Utilisateur

AJOUTER UN UTILISATEUR
Ajouter un nouvel utilisateur à votre système

Nom:

Prénom:

Description:

Email:

Mot de passe:

Date de naissance:

Role de l'utilisateur:

FIGURE 39 – La page d'ajout d'un utilisateur.

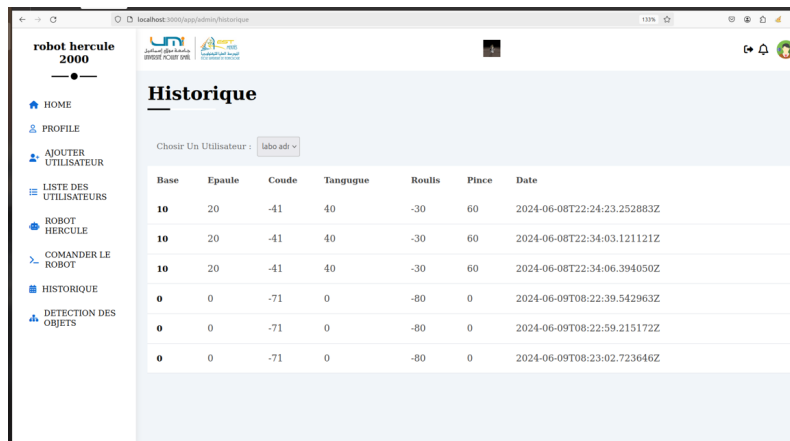
robot hercule 2000

Listes Utilisateurs

Nom Utilisateur	description	role	date_naissar	
labo admin Ladmin@gmail.com	AUCUN	Admin	1990-01-01	SUPPRIMER
labo2 admin2 Ladmin2@gmail.com	AUCUN	Admin	1990-01-01	SUPPRIMER
sdjdhdh hhhh hhhdhd@gmail.com	hhhh	Admin	2002-09-09	SUPPRIMER
chams Ediane chamsEdiane@gmail.com	AUCUN	User	1990-01-01	SUPPRIMER

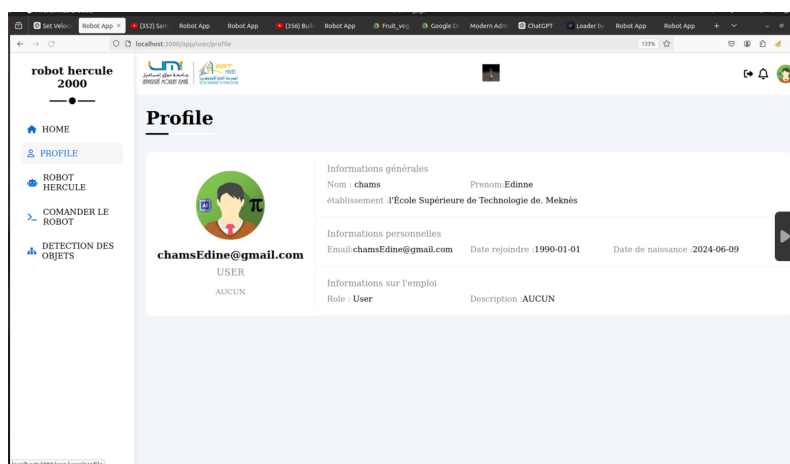
FIGURE 40 – La page de liste des utilisateurs

- Commander le robot : À partir de cette page, nous enverrons les valeurs des 6 degrés de liberté (base, épaule, coude, tangage, roulis, pince) au serveur pour effectuer les mouvements, que ce soit sur le robot réel ou dans ROS.



Base	Epaule	Coude	Tanguage	Roulis	Pince	Date
10	20	-41	40	-30	60	2024-06-08T22:24:23.252883Z
10	20	-41	40	-30	60	2024-06-08T22:34:03.121121Z
10	20	-41	40	-30	60	2024-06-08T22:34:06.394050Z
0	0	-71	0	-80	0	2024-06-09T08:22:39.542963Z
0	0	-71	0	-80	0	2024-06-09T08:22:59.215172Z
0	0	-71	0	-80	0	2024-06-09T08:23:02.723646Z

FIGURE 41 – Liste des commandes passées au robot



robot hercule 2000

Profile

chamsEdine@gmail.com
USER
AUCUN

Informations générales
Nom : chams
Prenom : Edinne
établissement : l'Ecole Supérieure de Technologie de Meknes

Informations personnelles
Email: chamsEdine@gmail.com
Date rejoindre : 1990-01-01
Date de naissance : 2024-06-09

Informations sur l'emploi
Role : User
Description : AUCUN

FIGURE 42 – Les informations personnelles de l'utilisateur connecté.

- Detection des objets : Dans cette page, nous allons exploiter le service de caméra développé en Django, basé sur un modèle de détection en CNN (Convolutional Neural Network).

4.5.3 Authentification et autorisation

Dans notre application React, nous avons configuré les routes pour séparer les interfaces admin et utilisateur. Les données de session, telles que le nom d'utilisateur et le rôle, sont stockées localement après la connexion. Ces données guident l'utilisateur vers la section appropriée de l'application. Cette approche garantit une navigation

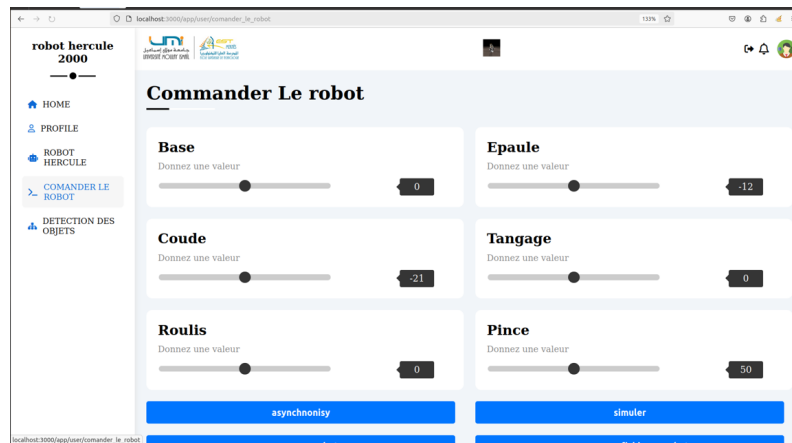


FIGURE 43 – Commander le robot :

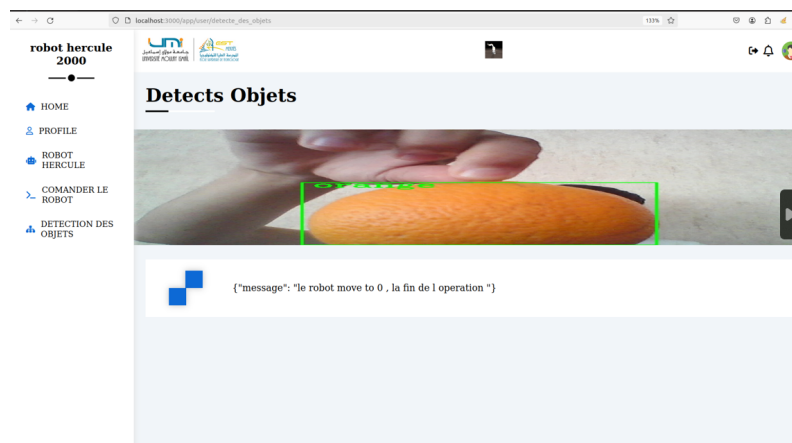


FIGURE 44 – Afficher le flux de la caméra et le résultat de détection.

intuitive et sécurisée.

Configuration des routes administratives :

Configuration des routes administratives des utilisateurs :

la configuration des routes Dans notre application :

```

const AdminRoutes = () => (
  <Routes>
    <Route index element={<Home />} />
    <Route path="utilisateur" element={<Utilisateur />} />
    <Route path="profile" element={<Profile />} />
    <Route path="liste_utilisateur" element={<ListeUtilisateur />} />
    <Route path="robot_hercule" element={<Robot />} />
    <Route path="comander_le_robot" element={<Comander />} />
    <Route path="detecte_des_objets" element={<DetecteObjets />} />
    <Route path="historique" element={<Historique />} />
  </Routes>
);

```

FIGURE 45 – Configuration des routes administratives :

```

const UserRoutes = () => (
  <Routes>
    <Route index element={<Home />} />
    <Route path="profile" element={<Profile />} />
    <Route path="robot_hercule" element={<Robot />} />
    <Route path="comander_le_robot" element={<Comander />} />
    <Route path="detecte_des_objets" element={<DetecteObjets />} />
  </Routes>
);

```

FIGURE 46 – Configuration des routes administratives des utilisateurs .

```

const AppRoutes = () => (
  <Routes>
    <Route path="/app" element={<App />} />
    <Route path="admin/*" element={<PrivateRouteAdmin><AdminRoutes /></PrivateRouteAdmin>} />
    <Route path="user/*" element={<PrivateRouteUser><UserRoutes /></PrivateRouteUser>} />
    <Route path="*" element={<Navigate to="/notfound" />} />
  </Routes>
  <Route path="/login" element={<Login />} />
</Routes>
);

export default AppRoutes;

```

FIGURE 47 – la configuration des routes Dans notre application .

5 Etat du projet et perspectives envisageables

Nous allons réussir à mettre en place un modèle virtuel du robot Hercule, ainsi qu'une application web avec deux aspects : le frontend et le backend pour commander le robot, que ce soit dans le monde virtuel de ROS ou dans le monde réel du robot physique. Et en plus, développer un modèle de détection des objets et exécuter des commandes prédéfinies pour prendre l'objet détecté et le déplacer à une position prédéfinie. Chaque objet aura sa propre position prédéfinie dans notre programme.

Nous souhaitons améliorer le code pour garantir une fiabilité totale de l'application, en prenant en compte tous les cas d'erreurs possibles et en rendant sa prise en main plus intuitive. En guise d'amélioration, il serait possible d'ajouter le calcul automatique des positions des objets détectés et de les saisir à l'aide d'un modèle géométrique direct et indirect. Cela permettrait au modèle virtuel de détecter les objets dans le rayon d'action d'Hercule et de calculer les trajectoires optimales pour les atteindre.

6 Difficultés rencontrées

Pendant la réalisation de ce projet, j'ai rencontré certaines difficultés. La principale difficulté était la prise en main des outils utilisés, car la plupart de la documentation était en anglais et la robotique est un domaine encore peu répandu. Cela a nécessité beaucoup d'heures de lecture de documentation et de visionnage de tutoriels avant de pouvoir utiliser ces outils correctement dans la réalisation de notre application. De plus, la conception de l'application et la gestion des mouvements en utilisant des modules de détection ont également posé des défis. Il est important de noter que l'accès au contenu nécessaire pour étudier le domaine de la robotique ou de la reconnaissance de formes est souvent payant.

Conclusion générale

J'ai pu mettre au point une application web pour superviser et commander un bras robotique à 6 degrés de liberté. De plus, cette application gère les utilisateurs et les mouvements du robot, et elle détecte également des objets à l'aide d'une caméra en utilisant un modèle basé sur l'apprentissage automatique. et aussi détecte des objets à partir d'une caméra à l'aide d'un modèle basé sur le machine learning.

A travers ce rapport, j'ai tenté de vous expliquer les différentes étapes que j'ai traversées afin d'accomplir ma mission avec succès. Je n'ai pas pu vous décrire tous les problèmes que j'ai rencontrés, car j'ai préféré me concentrer sur les problèmes majeurs qui ont eu un impact sur le résultat final.

Ce stage m'a permis de découvrir davantage le domaine de la robotique, qui a beaucoup progressé ces dernières années. J'ai également eu l'occasion d'apprendre sur l'apprentissage automatique, en particulier l'utilisation de réseaux de neurones convolutionnels (CNN) pour détecter des objets.

Ce stage a été une expérience très satisfaisante et enrichissante pour moi. Travailler dans de bonnes conditions et pouvoir appliquer mes connaissances sont des aspects positifs de cette expérience. Ce n'était pas seulement une opportunité d'apprentissage technique, mais aussi une expérience humaine, grâce aux personnes avec qui j'interagissais chaque jour. Elles ont rendu ce stage exceptionnel.

Webographie

[1] <https://wiki.ros.org/noetic>.

[2] <https://moveit.picknik.ai/main/index.html>.

[3] <https://wiki.ros.org/Sensors/Cameras>.

[4] <https://fr.mathworks.com/discovery/convolutional-neural-network.html>.

[5] <https://stanford.edu/shervine/cs-230/cheatsheet-convolutional-neural-networks>.

[6] <https://developer.mozilla.org/fr/docs/Learn/Server-side/Django>.

[7] https://en.wikipedia.org/wiki/Robot_Operating_System.

[8] <http://wiki.ros.org/rospy>.